

Andrew Sutton <asutton@lock3software.com>

Roger Orr <rogero@howzatt.demon.co.uk>

Gabriel Dos Reis <gdr@microsoft.com>

Target audience: EWG, CWG

2020-02-11

☐ Hide deleted text ☐ Hide inserted text

Disallow changing concept values

This paper resolves GB046 as "accepted with modifications"

The current Working Paper allows the satisfaction of a concept value to change a) within a translation unit and b) between translations. GB046 request to disallow this, by guaranteeing that all constraints instantiated with equivalent template arguments produce the same value. In particular, GB046 asks:

A program which causes a given set of template arguments to change their satisfaction of a constraint, and relies upon this change, is ill-formed, no diagnostic is required.

The proposed resolution better matches the design intent of concepts and its programming model. The result of instantiating a template should not depend on the source location where the instantiation was requested. Similarly, the satisfaction of a constraint should not depend on the location where the satisfaction was requested.

The change (from the Working Draft) impacts the performance of implementations. In particular, it allows an implementation to memoize the results of constraint satisfaction, as originally intended, and more specifically to retain those results for the duration of semantic analysis. The performance difference between non-caching implementations and caching implementations is significant: compile times can improve by 500% or more. Without the proposed resolution, performance necessarily degrades by 5-15%. (Performance results have reported informally by users and implementers. There is no benchmark.)

Additional notes on the caching strategies of GCC are presented at the end of this paper. They are included to support the proposed wording.

Some concerns have been expressed that future reflection and metaprogramming features could interact poorly with the proposed resolution. In particular, there is an intent to provide facilities that support queries and source code transformations that inherently depend on the source locations where they are invoked. This resolution would (at least for now) make it practically impossible to write e.g., concepts whose satisfaction on such queries. The proposed resolution is conservative in this regard. We could later relax this rule to facilitate potential future uses of reflection and metaprogramming.

Proposed wording

Add the following to [temp.constr.atomic] 13.5.1.2 to paragraph 3.

To determine if an atomic constraint is satisfied, the parameter mapping and template arguments are first substituted into its expression. If substitution results in an invalid type or expression, the constraint is not satisfied. Otherwise, the lvalue-to-rvalue conversion (7.3.1) is performed if necessary, and E shall be a constant expression of type bool. The constraint is satisfied if and only if evaluation of E results in true. **If, at different points in the program, the satisfaction result is**

different for identical atomic constraints and template arguments, the program is ill-formed, no diagnostic required.

Caching notes

We provide additional discussion on the caching approach of one implementation in order to motivate the proposed wording below.

GCC implements several layers of caching in its concepts implementation. These are:

- The results of normalization are cached for every concept and constrained declaration.
- The results of constraint satisfaction are cached for every constrained declaration.
- The results of substituting through each atomic constraint is cached.
- The results of subsumption are cached.

The caching of normalized constraints provides good performance benefits. The caching of subsumption results have had no observable impact on performance based on Casey Carter's cmcstl2. However, neither of these implementation techniques are salient to this discussion since the results of normalization does not change over the translation unit. The same is true for subsumption since it compares normalized constraints.

The result of caching satisfaction results for constrained declarations has little impact on performance - maybe 1% improvement over non-caching.

The result of caching atomic constraints yields (by far) the greatest performance benefit - 300-500% faster over non-caching. The single greatest cost of satisfying constraints was the instantiation of atomic constraints.

Currently, GCC clears its satisfaction caches on various events (e.g., the completion of a class). Adopting the proposed resolution may improve performance an additional 5-15%, possibly more.